FleetGraph

A Project Intelligence Agent for Ship

Background

Ship shows you what's happening. It doesn't tell you what's wrong.

Project teams drift. Issues go stale, sprints slip, standups don't get logged, blockers sit unresolved for days. The people responsible are busy, context-switching, and rarely looking at a dashboard when something goes wrong. The solution isn't a better dashboard. It's an agent that watches the system for them - and does something about it.

Your job is to build a project intelligence agent that reads the state of a project in Ship, reasons about what it finds, and makes the experience meaningfully better for the people using it. The agent should be **proactive**, meaning it should surface things nobody asked for. It should make decisions, not just fetch data. And it should know when to act and when to wait for a human.

Problem detection is one place to start. But don't stop there. Challenge yourself to identify every way a graph agent could make Ship more useful - not just for catching what's wrong, but for making the right next action obvious. The best implementations will find use cases we haven't thought of yet. Your job is to discover what this tool can do for a project team - and build it.

Your implementation must include a chat interface - but it must be embedded in context and scoped to what the user is looking at. A chat window on an issue should know about that issue. A chat window on a sprint should know about that sprint. It should feel like a power feature, not the primary interaction model. A standalone chatbot is not a graph agent.

The underlying goal is not to build a feature. It is to understand what problems a graph agent can solve that nothing else can - and prove it with working software running against real data.

Project Overview

One-week sprint with four deadlines:

Checkpoint	Deadline
Architecture Defense	4 hours after assignment
MVP	Tuesday, 11:59 PM
Early Submission	Thursday, 11:59 PM
Final Submission	**Sunday, Noon**

The Two Modes of FleetGraph

FleetGraph operates in two distinct modes. You must implement both.

Proactive - the agent pushes

The graph runs on its own schedule or in response to Ship events. It monitors project state, detects conditions worth surfacing, and delivers findings to the team without being asked. The agent decides when something is worth acting on and when to stay quiet.

On-Demand - the user pulls

The graph runs when a user invokes it from within the Ship interface. The chat interface is context-aware: it knows what the user is looking at and uses that as the starting point for its reasoning. The user asks a question or requests an action; the graph does the work.

Both modes run through the same graph architecture. The difference is the trigger, not the graph.

What the Agent Is Responsible For

You must define this. It is your first deliverable and lives in FLEETGRAPH.md.

Before writing any code, answer:

- What does this agent monitor proactively?
- What does it reason about when invoked on demand?
- What can it do autonomously?
- What must it always ask a human about before acting?
- Who does it notify, and under what conditions?
- How does it know who is on a project and what their role is?
- How does the on-demand mode use context from the current view?

This is a design problem. There is no prescribed answer. The quality of your agent's responsibility definition is graded as a primary deliverable.

Trigger Model

The proactive mode must run without a user present. How it does that is your decision to make and defend:

- Does it poll on a schedule? How frequently?
- Is it triggered by Ship events via webhook?
- Is it a hybrid of both?

There is no correct answer. There is a defensible one. Document your decision and its tradeoffs in FLEETGRAPH.md.

Observability

LangGraph traces automatically once these are set. If you are not using LangGraph, you are responsible for instrumenting your graph manually to produce equivalent traces.

You will submit shared observability trace links as part of every deliverable. Traces must demonstrate that the graph produces different execution paths under different conditions. A graph that looks identical across every run is a pipeline, not a graph.

MVP Requirements (Due Tuesday, 11:59 PM)

All items required to pass:

- ☐ Graph running with at least one proactive detection wired end-to-end
- ☐ LangSmith tracing enabled with at least two shared trace links submitted showing different execution paths
- ☐ FLEETGRAPH.md submitted with Agent Responsibility and Use Cases sections completed - at least 5 use cases defined
- ☐ Graph outline complete - node types, edges, and branching conditions documented in FLEETGRAPH.md
- ☐ At least one human-in-the-loop gate implemented
- ☐ Running against real Ship data - no mocked responses
- ☐ Agent chat and notifications are accessible in the UI
- ☐ Deployed and publicly accessible
- ☐ Trigger model decision documented and defended in FLEETGRAPH.md

---

Performance Requirements

Metric	Goal
Problem detection latency	< 5 minutes from event appearing in Ship to agent surfacing it
Cost per graph run	Documented and defended in FLEETGRAPH.md
Estimated runs per day	Documented and defended in FLEETGRAPH.md

Detection latency will be verified with a timed test run. An event will be introduced into Ship and the clock starts. The agent must surface it within the window.

Test Cases

You define your own test cases. For each use case in your use case document, provide:

- The Ship state that should trigger the agent
- What the agent should detect or produce
- The LangSmith trace from a run against that state

You own the test cases. The grader verifies that your agent does what you said it would do, given the conditions you defined. Document all test cases and trace links in FLEETGRAPH.md.

AI Cost Analysis

Development and Testing Costs

Track and report your actual spend:

- Claude API costs (input and output token breakdown)
- Number of graph agent invocations during development
- Total development spend

Production Cost Projections

Estimate monthly costs at scale:

100 Users	1,000 Users	10,000 Users
$___/month	$___/month	$___/month

Include assumptions: proactive runs per project per day, on-demand invocations per user per day, average tokens per invocation.

Deliverables

All final deliverables live in two files at the root of your repository:

File	Contents
PRESEARCH.md	Completed pre-search checklist
FLEETGRAPH.md	All sections below, filled in

FLEETGRAPH.md must contain the following completed sections at final submission:

Section	Due
Agent Responsibility	MVP
Graph Diagram	MVP
Use Cases	MVP
Trigger Model	MVP
Test Cases	Early Submission
Architecture Decisions	Early Submission
Cost Analysis	Final Submission

Constraints

- LangGraph is recommended; any other framework requires manual LangSmith instrumentation
- LangSmith tracing required from day one
- Chat interface must be embedded in context - no standalone chatbot pages

---

PRESEARCH

The goal is to make informed decisions about your agent's responsibilities and architecture.

Phase 1: Define Your Agent

1. Agent Responsibility Scoping

- What events in Ship should the agent monitor proactively?
- What constitutes a condition worth surfacing?
- What is the agent allowed to do without human approval?
- What must always require confirmation?
- How does the agent know who is on a project?
- How does the agent know who to notify?
- How does the on-demand mode use context from the current view?

2. Use Case Discovery (minimum 5)

- Think about the roles: Director, PM, Engineer
- For each use case define: role, trigger, what the agent detects or produces, what the human decides
- Do not invent use cases - discover pain points first

3. Trigger Model Decision

- When does the proactive agent run without a user present?
- Poll vs. webhook vs. hybrid - what are the tradeoffs?
- How stale is too stale for your use cases?
- What does your choice cost at 100 projects? At 1,000?

Phase 2: Graph Architecture

4. Node Design

- What are your context, fetch, reasoning, action, and output nodes?
- Which fetch nodes run in parallel?
- Where are your conditional edges and what triggers each branch?

5. State Management

- What state does the graph carry across a session?
- What state persists between proactive runs?
- How do you avoid redundant API calls?

6. Human-in-the-Loop Design

- Which actions require confirmation?
- What does the confirmation experience look like in Ship?
- What happens if the human dismisses or snoozes?

7. Error and Failure Handling

- What does the agent do when Ship API is down?
- How does it degrade gracefully?
- What gets cached and for how long?

Phase 3: Stack and Deployment

8. Deployment Model

- Where does the proactive agent run when no user is present?
- How is it kept alive?
- How does it authenticate with Ship without a user session?

9. Performance

- How does your trigger model achieve the < 5 minute detection latency goal?
- What is your token budget per invocation?
- Where are the cost cliffs in your architecture?

FLEETGRAPH.md

Submission template - fill in each section as you build

Agent Responsibility

Define what this agent monitors, what it reasons about, what it can do autonomously, what requires human approval, who it notifies and when, and how the on-demand mode uses context from the current view.

Graph Diagram

Provide a visual map of your graph covering both proactive and on-demand modes. Include all nodes, edges, and conditional branches. Submit either a LangGraph Studio screenshot (embedded as an image) or a Mermaid diagram as a code block.

Use Cases

Define at least 5 use cases. For each, provide: role, trigger, what the agent detects or produces, and what the human decides.

#	Role	Trigger	Agent Detects / Produces	Human Decides
1				
2				
3				
4				
5				

Trigger Model

Document your trigger model decision - poll, webhook, or hybrid. Explain the tradeoffs and defend your choice in terms of cost, reliability, and detection latency.

Test Cases

For each use case above, provide: the Ship state that should trigger the agent, what the agent should detect or produce, and the LangSmith trace link from a run against that state.

#	Ship State	Expected Output	Trace Link
1			
2			
3			
4			
5			

Architecture Decisions

Document your key architecture decisions and the tradeoffs you considered. Cover: framework choice, node design rationale, state management approach, and deployment model.

Cost Analysis

Development and Testing Costs

Item	Amount
Claude API - input tokens	
Claude API - output tokens	
Total invocations during development	
Total development spend	

Production Cost Projections

100 Users	1,000 Users	10,000 Users
$___/month	$___/month	$___/month

Assumptions:

- Proactive runs per project per day:
- On-demand invocations per user per day:

- Average tokens per invocation:
- Cost per run:
- Estimated runs per day: